

Apache Kafka

Aurreratua

Kafka klusterra, AEMET Bronze/Silver/Gold pipeline-a, NiFi integrazioa eta Kafka Connect.

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Aplikatua



01

2. Kasua — Kafka Klusterra

Kafka Kluster baten topologia

- **NODO 1 + BROKER 1** — garapena, probak, ariketa sinpleak
- **NODO 1 + BROKER ANITZ** — kluster baten portaera simulatzeko
- **NODO ANITZ + BROKER ANITZ** — produkzio-inguruneetan ohikoa

Docker Compose - 3 brokerreko klusterra

Broker 1
localhost:9092

Broker 2
localhost:9093

Broker 3
localhost:9094

Broker bakoitzak:

- Broker bakoitzak **ID propioa** dauka
- Broker bakoitzak **bere biltegitratzea** dauka
- Hirurak **Kafka kluster bereko kideak** dira
- Konfigurazioa **compose.yaml** fitxategian definituko da

— 3 BROKERREKO KLUSTERRA

Docker Compose – compose.yaml

```
compose.yaml
1  services:
2
3  # =====
4  # KAFKA BROKER 1
5  # =====
6  kafka1:
7    image: apache/kafka:latest
8    container_name: iabd-kafka1
9    hostname: kafka1
10
11    ports:
12      - "9092:9092"
13
14    environment:
15      KAFKA_NODE_ID: 1
16      KAFKA_PROCESS_ROLES: broker,controller
17
18      # KRaft Controller Quorum
19      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
20      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
21
22      # Listeners
23      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
24      KAFKA_LISTENERS: INTERNAL://:19092,CONTROLLER://:9093,EXTERNAL://:9092
25      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka1:19092,EXTERNAL://localhost:9092
26      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
27
28      # KRaft Cluster ID
29      CLUSTER_ID: 4L6g3nShT-eMctK--X86sw
30
31      # Kafka barneko topic-en erreplikazioa
32      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
33      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
34      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
35
36      # Kafka komandoak zuzenean erabiltzeko
37      PATH: "/opt/kafka/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

```
40  # =====
41  # KAFKA BROKER 2
42  # =====
43  kafka2:
44    image: apache/kafka:latest
45    container_name: iabd-kafka2
46    hostname: kafka2
47
48    ports:
49      - "9093:9092"
50
51    environment:
52      KAFKA_NODE_ID: 2
53      KAFKA_PROCESS_ROLES: broker,controller
54
55      # KRaft Controller Quorum
56      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
57      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
58
59      # Listeners
60      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
61      KAFKA_LISTENERS: INTERNAL://:19092,CONTROLLER://:9093,EXTERNAL://:9092
62      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka2:19092,EXTERNAL://localhost:9093
63      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
64
65      # KRaft Cluster ID
66      CLUSTER_ID: 4L6g3nShT-eMctK--X86sw
67
68      # Kafka barneko topic-en erreplikazioa
69      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
70      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
71      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
72
73      # Kafka komandoak zuzenean erabiltzeko
74      PATH: "/opt/kafka/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

```
77  # =====
78  # KAFKA BROKER 3
79  # =====
80  kafka3:
81    image: apache/kafka:latest
82    container_name: iabd-kafka3
83    hostname: kafka3
84
85    ports:
86      - "9094:9092"
87
88    environment:
89      KAFKA_NODE_ID: 3
90      KAFKA_PROCESS_ROLES: broker,controller
91
92      # KRaft Controller Quorum
93      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
94      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
95
96      # Listeners
97      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
98      KAFKA_LISTENERS: INTERNAL://:19092,CONTROLLER://:9093,EXTERNAL://:9092
99      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka3:19092,EXTERNAL://localhost:9094
100      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
101
102      # KRaft Cluster ID
103      CLUSTER_ID: 4L6g3nShT-eMctK--X86sw
104
105      # Kafka barneko topic-en erreplikazioa
106      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
107      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
108      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
109
110      # Kafka komandoak zuzenean erabiltzeko
111      PATH: "/ont/kafka/bin:/usr/local/sbin:/usr/local/bin:/usr/shin:/usr/bin:/shin:/hin"
```



Docker Compose – compose.yaml

```
114 # =====
115 # APACHE NIFI
116 # =====
117 nifi:
118   image: apache/nifi:latest
119   container_name: iabd-nifi-kafka
120   hostname: nifi
121
122   ports:
123     - "8443:8443"
124
125   environment:
126     SINGLE_USER_CREDENTIALS_USERNAME: nifi
127     SINGLE_USER_CREDENTIALS_PASSWORD: nifinifinifi
128
129   volumes:
130     - ./irteera:/opt/nifi/kafka-output
131
132   depends_on:
133     - kafka1
134     - kafka2
135     - kafka3
```

Kafka Listeners- Nondik konektatzen gara?

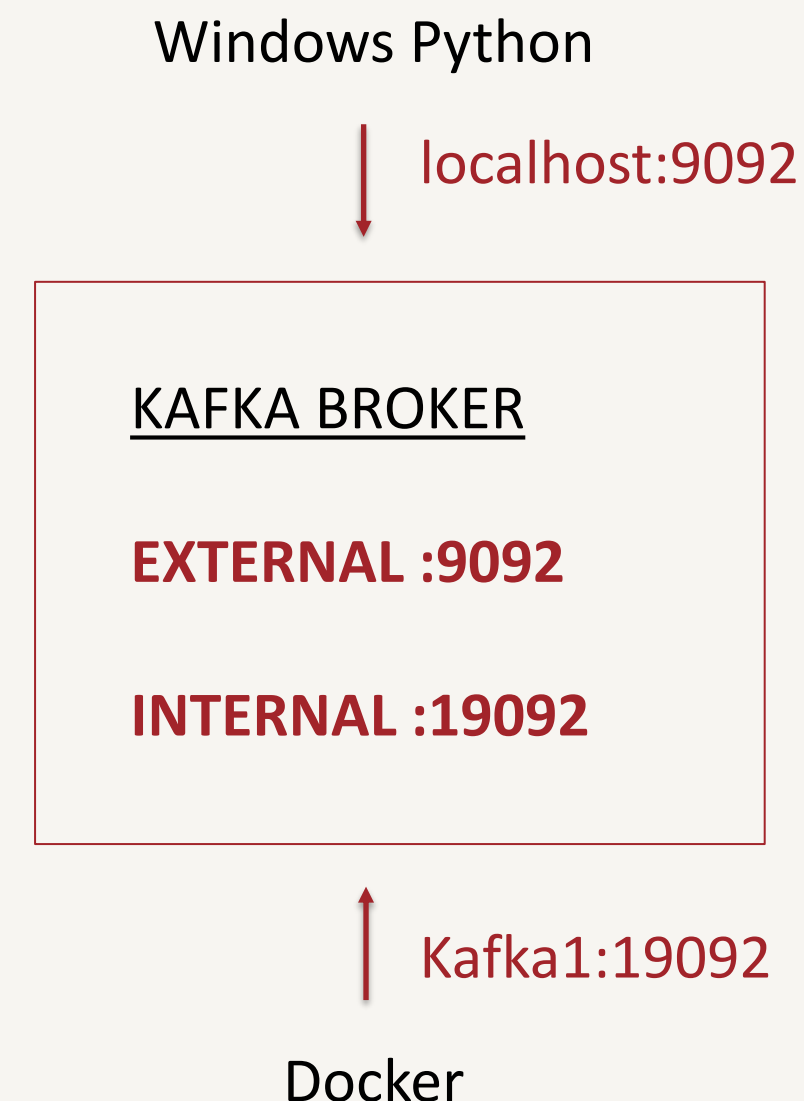
Listener batek Kafka broker-era nola eta nondik konektatu definitzen du.
Kafka broker batek listene bat baino gehiago izan ditzake.

Gure klusterrean 2 listener mota interesatzen zaizkigu:

- **EXTERNAL (9092)** → Docker-etik **KANPOKO** bezeroentzat.
Adib.: Python Windows-en.
- **INTERNAL (19092)** → Docker sare **BARRUKO** komunikaziorako.
Adib.: NiFi eta Kafka broker-ak.

ADI!

Bootstrap-server aukeratzeko, begiratu komandoa NON exekutatzen ari garen



— TOPIC-A: PARTIZIOAK + ERREPLIKAK

Topic-a sortzeko adib.: 3 partizio + 2 replication-factor

```
●●● topic-3p2r.sh bash

1 # Topic-a sortu - 3 partizio eta 2 errepikatze-faktore
2 kafka-topics.sh --create --topic iabd-topic-3p2r \
3   --bootstrap-server kafka1:19092 \
4   --partitions 3 \
5   --replication-factor 2
6
7 # Topic-a deskribatu
8 kafka-topics.sh --describe --topic iabd-topic-3p2r \
9   --bootstrap-server kafka1:19092
10
11 # OHARRAK:
12 #   --partitions 3          -> topic-a 3 partiziotan banatu
13 #   --replication-factor 2  -> partizio bakoitzaren 2 kopia
14 #   --bootstrap-server     -> klusterrarekin lehen konexioa egiteko bróker bat
15 #   3 broker-eko klusterrean RF=2 -> 3 broker behar lukete
16 #                               (RF <= broker-kopurua araua)
17 #   Partizio bakoitzak Leader 1 + Follower 1 izango ditu.
```

Topic-a deskribatu

●●● describe-3p2r.txt

text

```
1 Topic: iabd-topic-3p2r PartitionCount: 3 ReplicationFactor: 2
2
3 Topic: iabd-topic-3p2r Partition: 0 Leader: 1 Replicas: 1,2 Isr: 1,2
4 Topic: iabd-topic-3p2r Partition: 1 Leader: 2 Replicas: 2,3 Isr: 2,3
5 Topic: iabd-topic-3p2r Partition: 2 Leader: 3 Replicas: 3,1 Isr: 3,1
6
7 # INTERPRETAZIOA:
8 #   3 partizio x 2 kopia = 6 replika guztira
9 #   Broker bakoitzak 2 replika ditu (6/3=2)
10 #
11 #   Partizio 0: Leader: Broker 1, Follower: Broker 2
12 #   Partizio 1: Leader: Broker 2, Follower: Broker 3
13 #   Partizio 2: Leader: Broker 3, Follower: Broker 1
14 #
15 #   Leader-ak broker desberdinen artean banatuta daude (lan-zama orekatua)
16 #   ISR = une honetan sinkronizatuta dauden erreplikak
17 #   ISR = Replicas -> erreplika guztiak sinkronizatuta
```

Producer-a klusterrera konektatu

●●● producer-cluster.py

python

```
1 from kafka import KafkaProducer
2 from json import dumps
3 import time
4
5 # Producer-a HIRU broker-en bootstrap-listarekin
6 producer = KafkaProducer(
7     value_serializer=lambda v: json.dumps(v).encode('utf-8'),
8
9     bootstrap_servers=[
10         'localhost:9092',    # broker 1
11         'localhost:9093',    # broker 2
12         'localhost:9094'])   # broker 3
13
14 # 10 mezu round-robin partition-banaketa (gakorik gabe)
15 for i in range(10):
16     producer.send("iabd-topic-3p2r",
17         value={"izena": "ekoizlea " + str(i)})
18
19 time.sleep(1)    # buffer-bidaltzeko denbora
20 producer.flush() # ziurtatu mezu guztiak bidalita
```

Consumer-a klusterrera konektatu

●●● consumer-cluster.py

python

```
1 from kafka import KafkaConsumer
2 from json import loads
3
4 consumer = KafkaConsumer(
5     'iabd-topic-3p2r',
6     auto_offset_reset='earliest',
7     enable_auto_commit=True,
8     group_id='iabd-taldea-1',
9     value_deserializer=lambda v: json.loads(v.decode('utf-8')),
10    bootstrap_servers=[
11        'localhost:9092',
12        'localhost:9093',
13        'localhost:9094'])
14
15 # Partizio, offset eta gakoak erakutsi
16 for m in consumer:
17     print(f"P:{m.partition} O:{m.offset} K:{m.key} V:{m.value}")
```

Producer-ak key-rik erabiltzen ez badu:

●●● output-no-key.txt

text

```
1 P:1 O:0 K:None V:{'izena': 'ekoizlea 0'}
2 P:1 O:1 K:None V:{'izena': 'ekoizlea 3'}
3 P:2 O:0 K:None V:{'izena': 'ekoizlea 1'}
4 P:2 O:1 K:None V:{'izena': 'ekoizlea 5'}
5 P:0 O:0 K:None V:{'izena': 'ekoizlea 2'}
6 P:0 O:1 K:None V:{'izena': 'ekoizlea 4'}
7
8 # INTERPRETAZIOA:
9 #   K: None -> mezuak ez du key-rik
10 #   Mezuak partizio desberdinetan egon daitezke
11 #   Offset-a PARTIZIO BAKOITZAREN barruan independentea da:
12 #   Partition 0 -> O:0, O:1...
13 #   Partition 1 -> O:0, O:1...
14 #   Partition 2 -> O:0, O:1...
15 #   Ez dago ordena globalik
16 #   PARTIZIO BAKOITZAREN BARRUAN ordena mantentzen da
```

Key bera → mezu guztiak partizio berera

... producer-key.py

python

```
1 # Producer-aren SEND gakoarekin
2 producer.send("iabd-topic-3p2r",
3     value={"izena": "ekoizlea " + str(i)},
4     key=b"iabd") # gako bera mezu guztietan
5
6 # CONSUMER-EKO IRTEERAREN ADIBIDEA:
7 # P:0 O:3 K:b'iabd' V:{'izena': 'ekoizlea 0'}
8 # P:0 O:4 K:b'iabd' V:{'izena': 'ekoizlea 1'}
9 # P:0 O:5 K:b'iabd' V:{'izena': 'ekoizlea 2'}
10 # P:0 O:6 K:b'iabd' V:{'izena': 'ekoizlea 3'}
11 # ...
12 # P:0 O:12 K:b'iabd' V:{'izena': 'ekoizlea 9'}
13
14 # INTERPRETAZIOA:
15 # Key bera -> partizio bera
16 # Partizio zehatza (P0, P1 edo P2) producer-ak zehazten du.
17 # Offset-ak gorantz doaz partizio horren barruan
18 # Kafka-k partizio baten barruko mezuen ordena mantentzen du
```

Kafka Klusterra – ideia nagusiak

- Kluster batek **broker anitz** izan ditzake
- Topic-ak **partiziotan** banatzen dira
- Partizio bakoitzak **Leader** bat dauka
- **Replication Factor**-ak partizio bakoitzaren kopia kopurua adierazten du
- **ISR** -> sinkronizatuta dauden erreplikak
- Producer/Consumer-ek **bootstrap_servers** bidez klusterra aurkitzen dute
- **Key** bera -> **partizio bera** -> **ordena mantendu**

2. KASUA: Kafka Klusterra + Python + NiFi

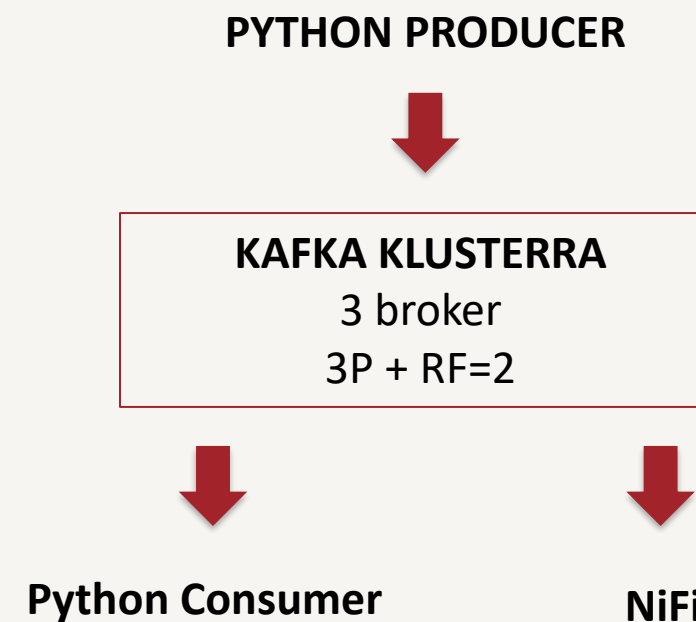
EGOERA

Sortu 3 brokerreko Kafka kluster bat Docker Compose erabiliz.

Sortu topic bat:

- 3 partizio
- Replication Factor = 2

Ondoren, Kafka topic bera hiru tresnarekin erabiliko dugu:



2. KASUA: Kafka Klusterra + Python + NiFi

EGINKIZUNAK

1. Kafka klusterra

- Sortu **compose.yaml** 3 brokerrekin eta NiFi-rekin eta abiarazi.

2. Topic-a

- Sortu topic bat: **3 partizio + RF=2** **ADI! Zein Bootstrap-server erabili?**

Komando hau Kafka container-aren barruan exekutatzen ari gara.

Beraz, Docker sare barruko listener-a erabili behar dugu: **kafka1:19092**

- Aztertu **--describe** erabiliz: **Leader, Replicas** eta **ISR**

3. Python Producer

- Faker erabiliz, sortu **10 pertsona**
- Bidali pertsona bakoitza Kafka mezu batean
- Datuak: *izena, adina, hiria, emaila*

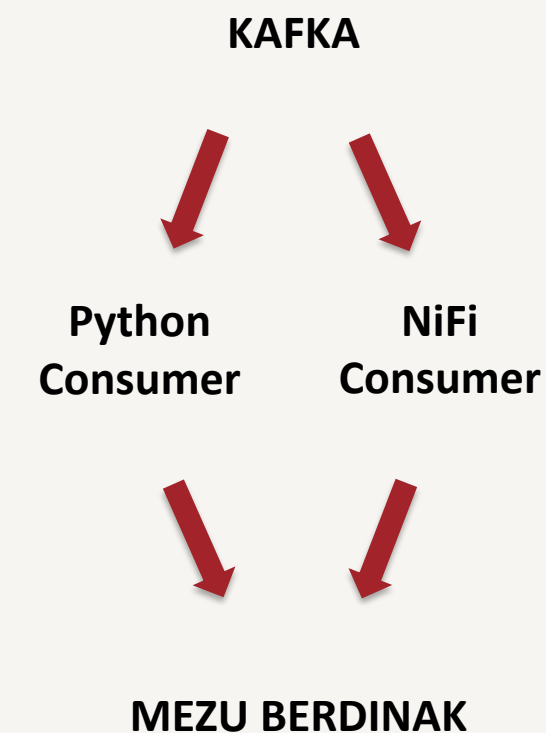
4. Python Consumer

- Kontsumitu topic-a eta erakutsi: **Partition | Offset | Key | Value**

5. NiFi Consumer

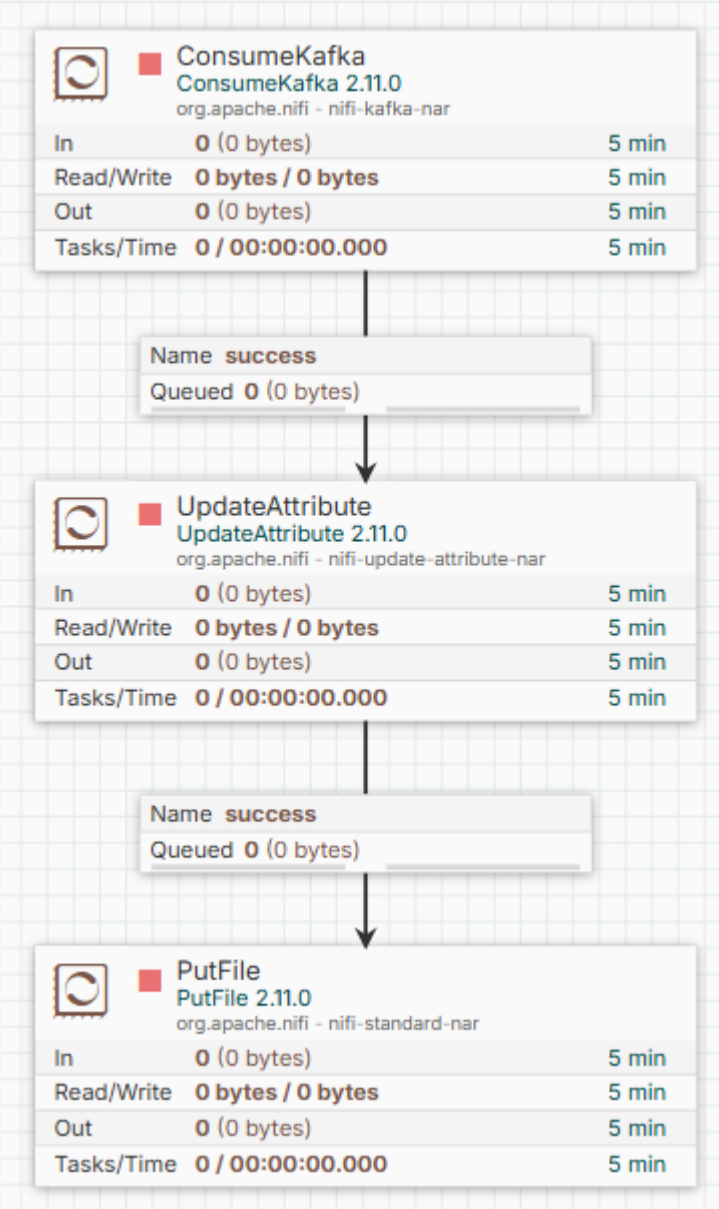
- Kontsumitu **topic bera** NiFi-tik (*ConsumeKafka* prozesadorea)
- Erabili Python Consumer-aren **consumer group desberdina**

6. Egiaztatu:



2. KASUA: Kafka Klusterra + Python + NiFi

NIFI-ko fluxua:



02

3. Kasua — AEMET

Bronze/Silver/Gold

— ZER EGINGO DUGU?

AEMET – Bronze / Silver / Gold

HELBURUA: eguraldi-datuak hainbat fasetan prozesatzea, jeruza bakoitza Kafka topic baten bidez konektatuz.

- **Bronze** – datu gordinak
- **Silver** – datu garbi eta egituratuak
- **Gold** – datu agregatuak / KPlak

Datuak pixkanaka prozesatu eta analisirako prestatuko dira.

Eguraldi API-a



BRONZE



SILVER



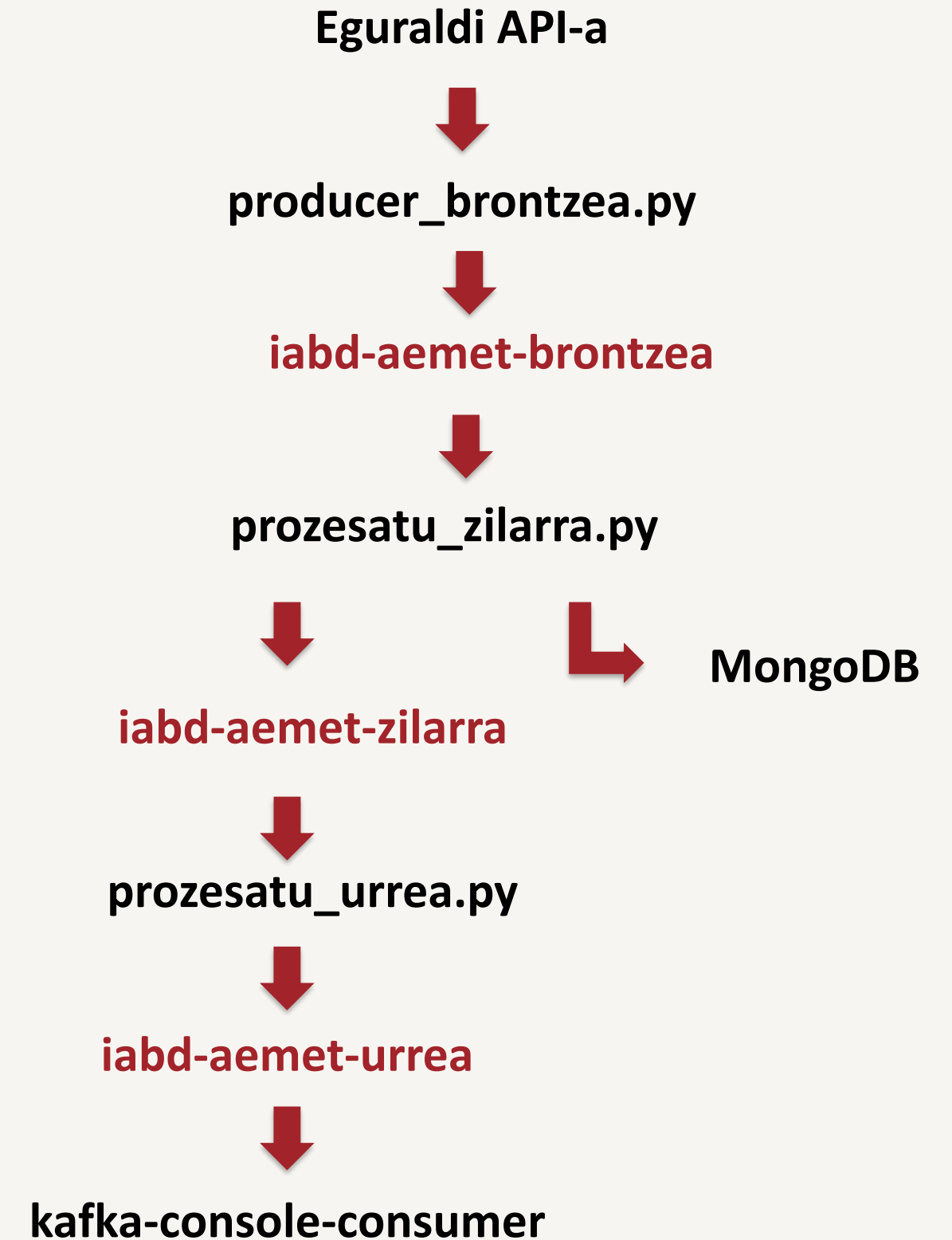
GOLD

AEMET – Bronze / Silver / Gold

Pipeline-a script independentez osatuta dago.

Pentsatu: script bakoitza zer da?

- Producer?
- Consumer?
- Consumer + Producer?



Hiru topic-ak sortu (Bronze, Silver, Gold)

Aurreko kasuan erabilitako Kafka klusterra erabili.

Sortu hiru topic:

- **iabd-aemet-brontzea**
- **iabd-aemet-zilarra**
- **iabd-aemet-urrea**

Topic-ak sortzeko: **kafka-topics.sh --create ...**

Egiaztatzeko: **kafka-topics.sh --list ...**

Bronze producer-a sortu: producer_brontzea.py

Programak honako hau egin behar du:

1. Eguraldi API-ari eskaera egin.
 - APIa: <https://api.el-tiempo.net/json/v3/provincias/03/municipios/03065>
2. JSON erantzuna jaso.
3. JSON osoa **iabd-aemet-brontzea** topic-era bidali.
4. 10 segundo itxaron.
5. Prozesua etengabe errepikatu.

Pistak: `requests.get(...)`
`KafkaProducer(...)`
`producer.send(...)`
`time.sleep(...)`

Zer espero dugu Bronze-n?

API-ak JSON nahiko handia itzuliko du.

Adibidez:

```
{
  "municipio": {
    "NOMBRE": "Elche/Elx",
  },
  "temperatura_actual": "21",
  "humedad": "55",
  ...
}
```

BRONZE-n ez dugu:

- eremurik kentzen
- datu-motarik aldatzen
- agregaziorik egiten

Datu gordina gordetzen dugu.

Bronze egiaztatzeko:

- bronze topic-a kontsumitu terminal batetik:
`kafka-console-consumer.sh --topic iabd-aemet-brontzea --from-beginning ...`

Sortu prozesatu_zilarra.py

prozesatu_zilarra.py programak honako hau egin behar du:

1. **iabd-aemet-brontzea** topic-a kontsumitu
2. JSON-etik beharrezko eremuak atera.
3. Silver objektu berria sortu.
4. **iabd-aemet-zilarra** topic-era bidali.
5. Silver datua MongoDB-n gorde.

Silver egitura

Silver objektuak egitura hau izan behar du: {

```
{
  "data": "...",
  "hiria": "...",
  "tenperatura": 0.0,
  "hezetasuna": 0.0
}
```

Baldintzak:

- **data:** uneko data eta ordua
- **hiria:** API-ko udalerriaren izena
- **tenperatura:** float
- **hezetasuna:** float

Pistak: `KafkaConsumer(...)`
`Json.loads(...)`
`Datetime.now(...)`
`float(...)`
`KafkaProducer(...)`
`MongoClient(...)`

iabd-aemet-brontzea

Consumer

TRANSFORMAZIOA

Producer

iabd-aemet-zilarra

MongoDB

Pentsatu: Consumer-a da? Producer-a da? Biak?

Silver egiaztatzeko:

- KAFKA: silver topic-a kontsumitu terminal batetik:
`kafka-console-consumer.sh --topic iabd-aemet-silver...`
- MONGODB: `use iabd`
`db.aemet_zilarra.find()`

Sortu prozesatu_urrea.py

prozesatu_urrea.py programak honako hau egin behar du:

1. **iabd-aemet-zilarra** topic-a kontsumitu.
2. Silver mezuak zerrenda batean Bildu.
3. 10 neurketa jasotakoan Pandas DataFrame bat sortu.
4. Estatistikak kalkulatu:
 - Batez_besteko_temperatura
 - Temperatura_min
 - Temperatura_max
 - Batez_besteko_hezetasuna
5. Gold objektua sortu.
6. **iabd-aemet-urrea** topic-era bidali.
7. Hurrengo 10 neurketekin berriro hasi.

Pistak:

- Pandas inportatu: `import pandas as pd`
- Silver mezuak zerrenda batean bildu: `mezuak = []`
- 10 mezu bildu diren egiaztatu: `if len(mezuak) == 10:`
- Zerrenda Pandas DataFrame bihurtu: `df = pd.DataFrame(mezuak)`
- Estatistikak kalkulatu:
 - Batezbestekoa: `.mean()`
 - Min: `.min()`
 - Max: `.max()`

Kontuan izan:

- 10 neurketa → Gold mezu 1

Zer espero dugu Gold-en?

10 neurketetatik kalkulatutako emaitza.

Adibidez:

```
{
  "hiria": "Elche/Elx",
  "neurketak": 10,
  "batez_besteko_tenperatura": 20.8,
  "tenperatura_min": 19.0,
  "tenperatura_max": 22.0,
  "batez_besteko_hezetasuna": 57.4
}
```

Gold egiaztatzeko:

- gold topic-a kontsumitu terminal batetik:
`kafka-console-consumer.sh --topic iabd-aemet-urrea --from-beginning ...`

Egiaztatu:

- Hasieran ez da Gold mezurik berehala agertzen.
- 10 Silver neurketa bildu behar dira.
- Ondoren Gold agregazioa agertu behar da.

Pipeline osoa martxan jarri

Ireki hainbat terminal.

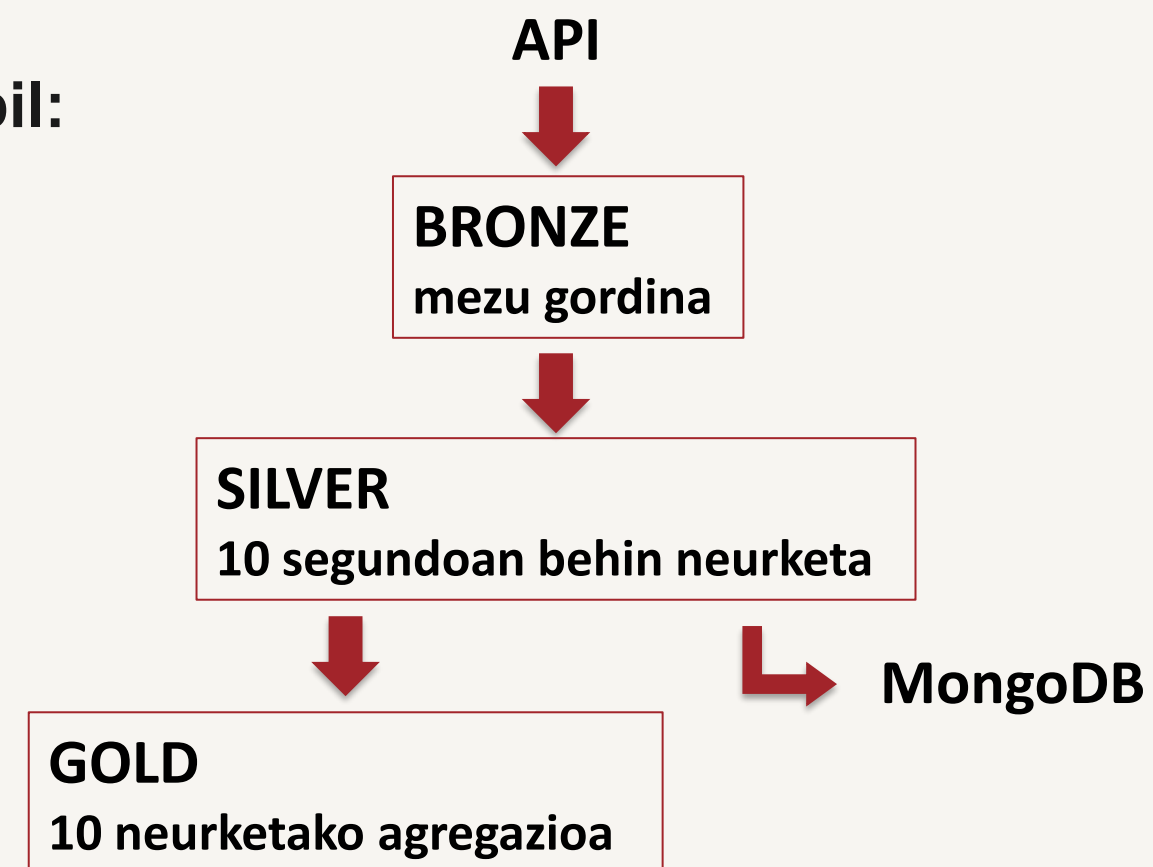
Exekuzio-ordena gomendatua:

1. Gold prozesua: `prozesatu_urrea.py`
2. Silver prozesua: `prozesatu_zilarra.py`
3. Bronze producer-a: `producer_brontzea.py`

Zergatik ordena honetan?

Consumer-ak prest egongo dira Producer-a datuak bidaltzen hasi baino lehen.

Dena ondo badabil:



Egiaztatu:

- Bronze-topic-ak mezuak ditu.
- Silver topic-ak mezuak ditu.
- MongoDB-k Silver dokumentuak ditu.
- Gold topic-ak agregazioak ditu.

— CONSUMER GROUP

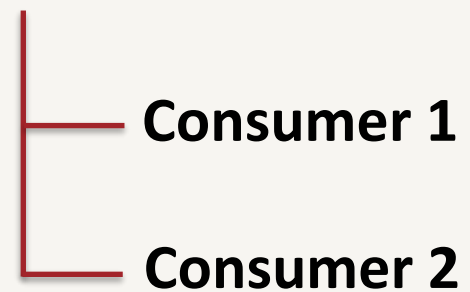
Consumer Group-ak berrikusi

Gogoratu:

GROUP ID BERA:

Consumer group bereko consumer-ek lana banatzen dute.

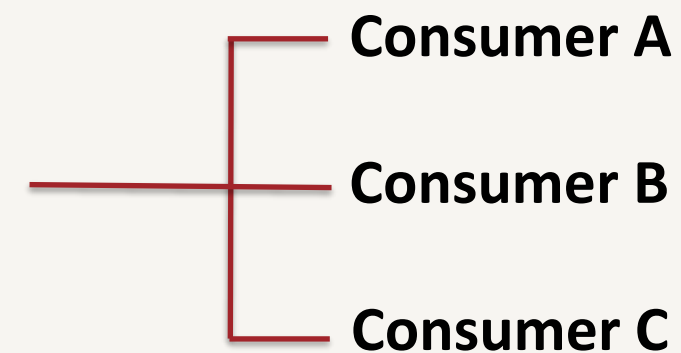
Topic



GROUP ID DESBERDINA:

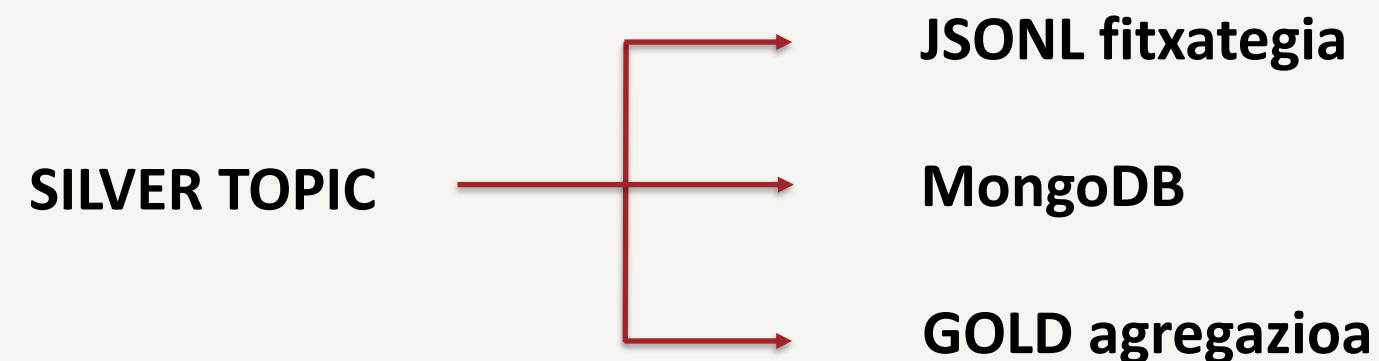
Consumer group bakoitzak mezu guztiak modu independentean irakur ditzake.

Topic



3. Kasua — Silver kontsumitzailea banatu

Orain Silver topic-eko datuak hiru helburu independente izango ditu:



Sortu hiru script independente:

- kontsumitzaileZilarraFitxategia.py
- kontsumitzaileZilarraMongoDB.py
- kontsumitzaileZilarra_EkoizleaUrrea.py

1. consumer independentea: kontsumitzaileZilarraFitxategia.py

Programak egin behar duena:

- **iabd-aemet-zilarra** kontsumitu.
- Mezu bakoitza fitxategi batean gorde:
 - Fitxategia: **silver_datuak.jsonl**
 - Mezu bakoitzak lerro bat okupatu behar du.

Consumer Group: **iabd-3kasua-fitxategia**

Pista: `open(..., "a")`
`json.dumps(...)`

2. consumer independentea: kontsumitzaileZilarraMongoDB.py

Programak egin behar duena:

- **iabd-aemet-zilarra** kontsumitu.
- Mezu bakoitza MongoDB-n gorde:

Consumer Group: **iabd-3kasua-mongo**

Egiaztatu: `db.aemet_zilarra.find(...)`

3. consumer independentea: kontsumitzaileZilarra_EkoizleaUrrea.py

Programak egin behar duena:

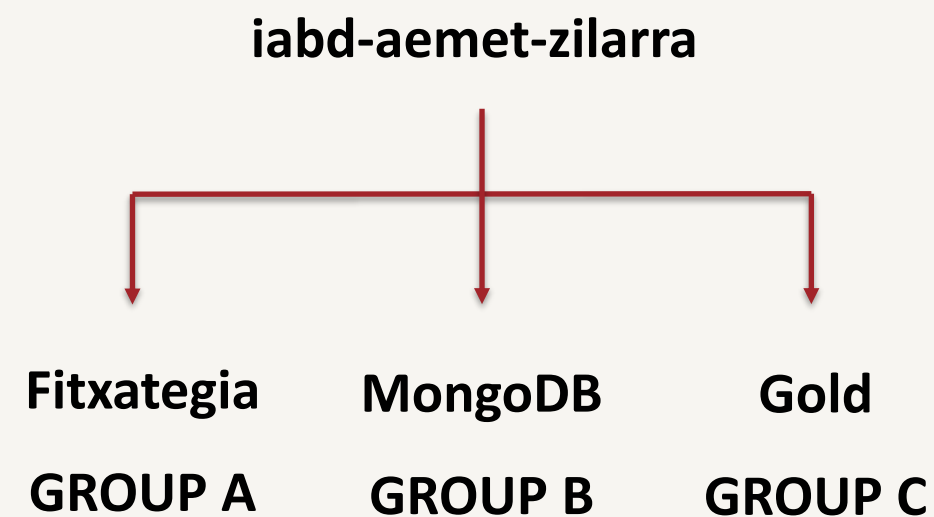
- **iabd-aemet-zilarra** kontsumitu.
- 10 mezu bildu.
- Pandas erabiliz agregazioa egin.
- Eraitza **iabd-aemet-urrea** topic-era bidali.

Consumer Group: **iabd-3kasua-gold**

Zergatik group ID desberdinak?

Hiru script-ek Silver mezu **guztiak** jaso behar dituzte.

Beraz:



Galdera: Zer gertatuko litzateke hirurek hau erabiliko balute? `group_id="iabd-3kasua"`

Pentsatu consumer group-en funtzionamenduan.

Erronka egiaztatu

Hiru script-ak aldi berean exekutatu.

Egiaztatu:

- **JSONL:** `silver_datuak.jsonl` datuak jasotzen ari da.
- **MongoDB:** `db.aemet_zilarra.find()` dokumentuak agertzen dira.
- **Gold:** Gold topic-ean agregazioak agertzen dira.

Consumer group-ak: `kafka-consumer-groups.sh --list --bootstrap-server...`

Hiru group ID desberdin agertu behar dira.

Azken hausnarketa

Erantzun:

1. Zer desberdintasun dago Bronze, Silver eta Gold artean?
2. Zergatik erabiltzen dugu topic bat geruza bakoitzaren artean?
3. **prozesatu_zilarra.py** Producer-a da ala Consumer-a da?
4. Zer abantaila du Bronze datuak gordetzeak?
5. Zer gertatzen da consumer bat gelditzen bada eta berriro abiarazten bada?
6. Zertarako erabiltzen da **group_id**?
7. Zergatik behar dituzte azken erronkako hiru consumer-ek Group ID desberdinak?
8. Zein abantaila du prozesua script independenteetan banatzeak?

Zer ikasi dugu?

API → Kafka → Python → Kafka → Python → Kafka

- Kafka topic-ak pipeline-etapak konektatzeko erabili ditugu.
- Producer eta Consumer-ak konbinatu ditugu.
- Bronze → Silver → Gold eredua aplikatu dugu.
- Python-ekin datuak eraldatu ditugu.
- Pandas-ekin agregazioak egin ditugu.
- MongoDB-n Silver datuak gorde ditugu.
- Consumer group-en portaera praktikan aplikatu dugu.
- Topic bera hainbat helburutarako kontsumitu dugu.

Kafka-k zerbitzu independenteak datu-fluxu baten bidex konektatzeko aukera ematen du.

03

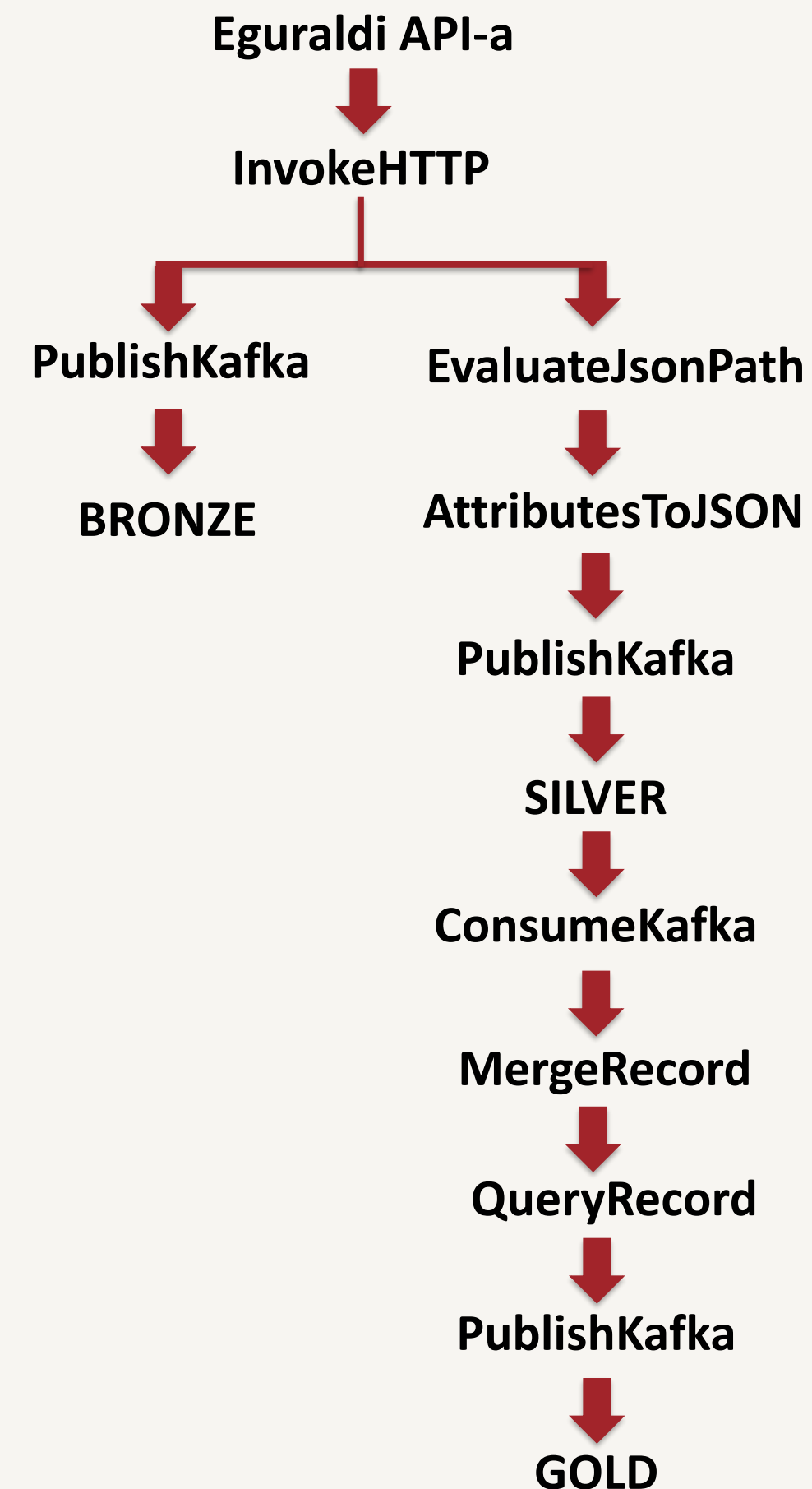
4. Kasua — NiFi + Kafka

— ZER EGINGO DUGU?

AEMET NiFi-rekin

HELBURUA: aurreko kasuko datu-pipeline bera beste tresna batekin eraikitzea.

- **Bronze** – datu gordina
- **Silver** – datu garbia
- **Gold** – datu agregatua



REST API → Bronze

- **Sortu fluxua:** InvokeHTTP → PublishKafka
- **API-a:** <https://www.el-tiempo.net/api/json/v2/provincias/03/municipios/03065>
- **Topic-a:** **iabd-aemet-brontzea**

Pistak:

- **InvokeHTTP:**
HTTP Method: GET
Remote URL: API URL
- **PublishKafka:**
Kafka Connection Service
Topic Name

REST API → Silver

- **InvokeHTTP-tik bigarren adarra sortu:** InvokeHTTP → EvaluateJsonPath → AttributesToJSON → PublishKafka
- **Silver-en behar ditugun eremuak:**
 - hiria
 - Temperatura
 - hezetasuna

Pistak:

- **EvaluateJsonPath:**
 - hiria: \$.municipio.NOMBRE
 - temperatura: \$.temperatura_actual
 - Hezetasuna: \$.humedad
 - Destination : flowfile-attribute
- **AttributesToJSON:**
 - Attributes List: hiria,temperatura,hezetasuna
 - Destination : flowfile-content

Silver Kafka-tik kontsumitu

- **Osatu:** AttributesToJSON → PublishKafka → **iabd-aemet-zilarra**
- **Egiaztatu bi topic-ak:**
 - **iabd-aemet-brontzea** → JSON osoa
 - **iabd-aemet-zilarra** → hiria + tenperatura + hezetasuna
- **Beste Process Group batean:** **iabd-aemet-zilarra** → ConsumeKafka

Pistak:

- **ConsumeKafka:**
Kafka Connection Service
Topics: iabd-aemet-zilarra
Group ID: iabd-4kasua-gold

10 neurketa bildu

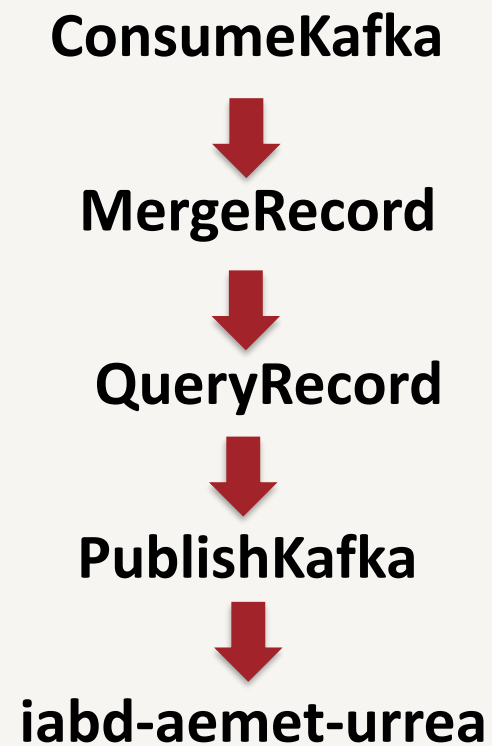
- **10 Silver neurketa FlowFile bakarrean bildu: ConsumeKafka → MergeRecord**
 - Record Reader → JsonTreeReader
 - Record Writer → JsonRecordSetWriter
 - Minimum Number of Records → 10
 - Maximum Number of Records → 10
- **Gold agregazioa: MergeRecord → QueryRecord**

Kalkulatu SQL-an:

 - batez_besteko_temperatura
 - temperatura_min
 - temperatura_max
 - batez_besteko_hezetasuna

Gold Kafka-ra bidali

- Osatu fluxua:



- Egiaztatu:

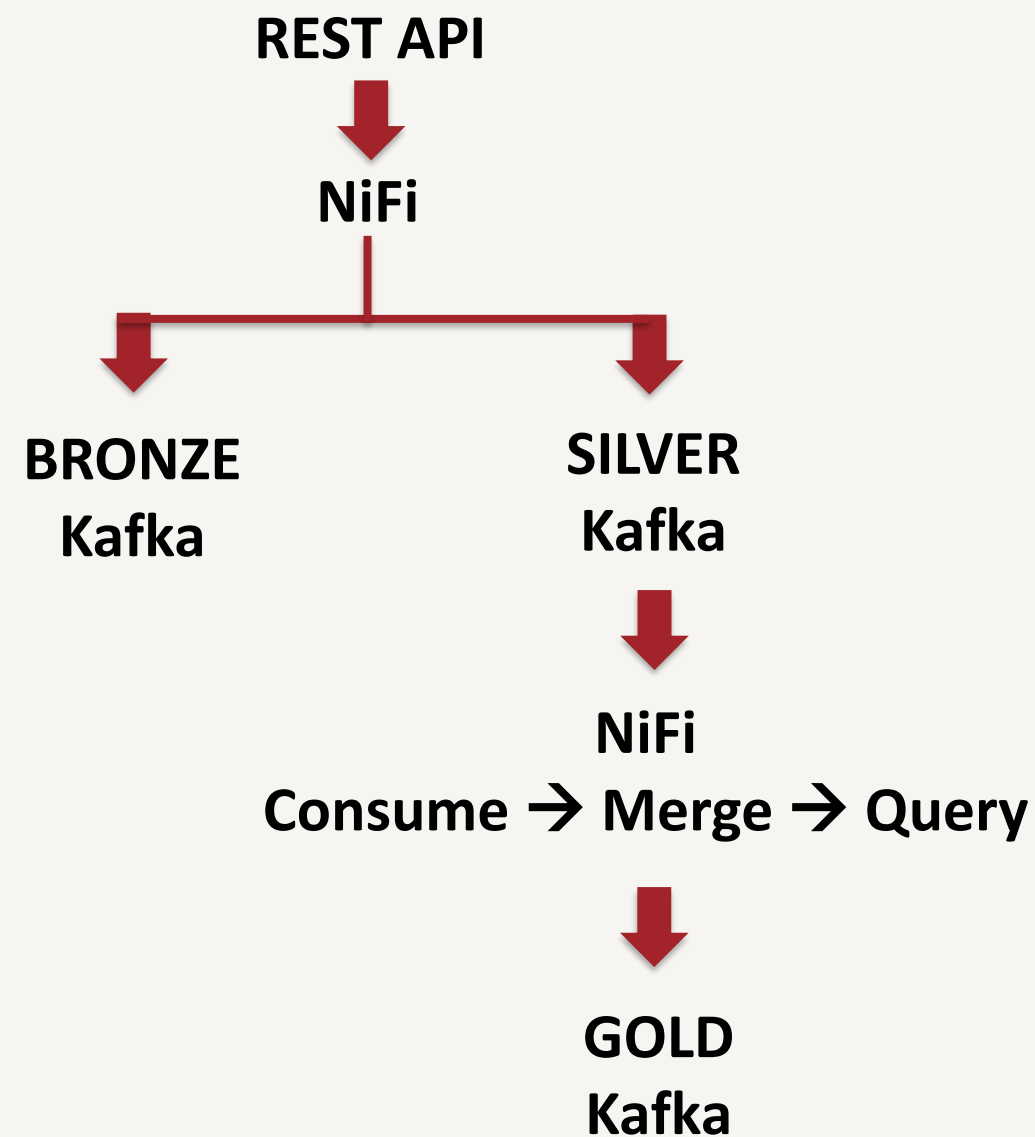
kafka-console-consumer.sh --topic iabd-aemet-urreea...

- Espero den antzeko emaitza:

```
{
  "hiria": "Elche/Elx",
  "batez_besteko_tenperatura": 20.8,
  "tenperatura_min": 19.0,
  "tenperatura_max": 22.0,
  "batez_besteko_hezetasuna": 57.4
}
```

Azken egiaztapena eta hausnarketa

- Amaierako pipeline-a:



- Hausnarketa:

1. Zer egiten du InvokeHTTP-k?
2. Zer desberdintasun dago Bronze eta Silver artean?
3. Zertarako erabiltzen dugu MergeRecord?
4. Zer egiten du QueryRecord-ek?
5. Zer baliokide dira 3. kasuko Pandas kalkuluak 4. kasuan?
6. Zer desberdintasun ikusten duzu Python eta NiFi bidezko implementazioen artean?

04 Kafka Connect

Kafka Connect

Kafka Connect

Kafka eta kanpoko sistemen artean datuak mugitzeko framework-a da.

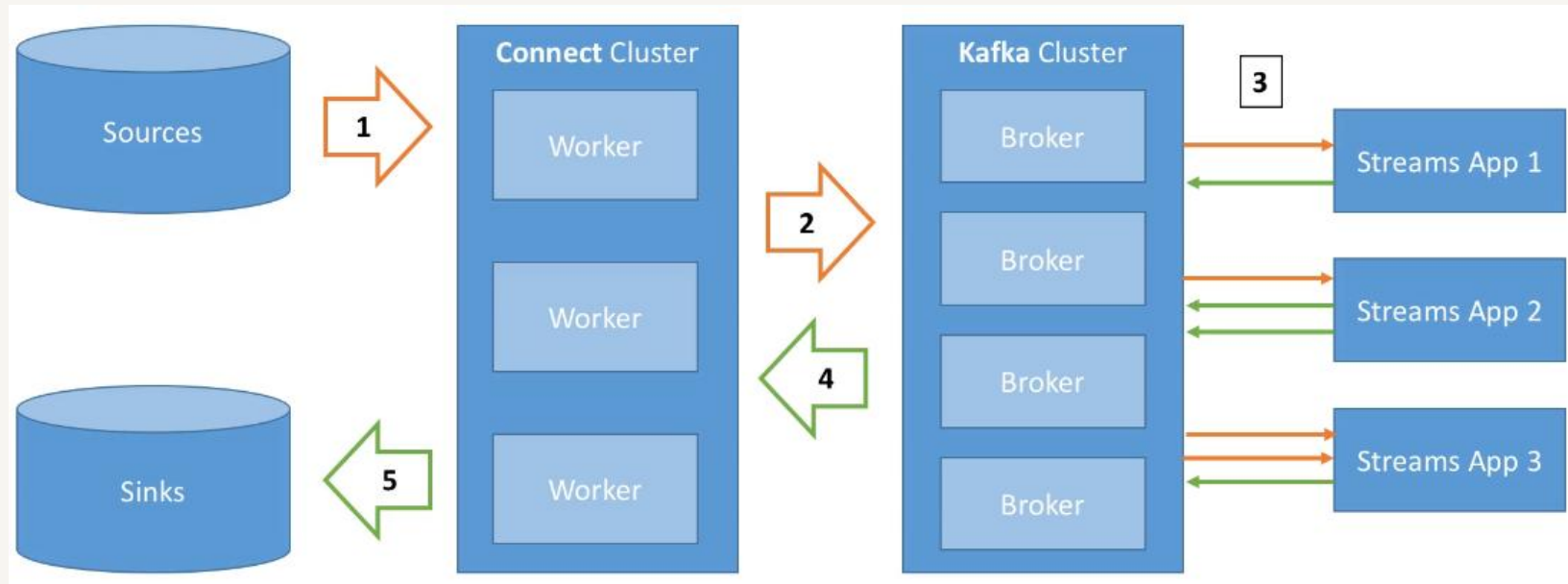
- **Source Connector** → kanpoko sistematik **Kafka-ra**
- **Sink Connector** → **Kafka-tik** kanpoko sistemara
- Konektoreak **konfigurazio bidez** definitzen dira.
- MySQL, MongoDB, Elasticsearch, S3... bezalako sistemetarako konektoreak daude.

KODE GABE

Ez dugu producer/consumer kode propioa idatzi behar integrazio arruntetarako.

— ARKITEKTURA

Kafka Connect — source eta sink konektoreak



- **1-2. SOURCE FLUXUA:** Kanpoko sistema → Connect → Kafka
- **4-5. SINK FLUXUA:** Kafka → Connect → kanpoko sistema

- **SOURCES:** source-systems (MySQL, MongoDB, FTP, S3, REST APIs...)
- **SOURCE CONNECTOR** — datuak Kafka-ra sartzen ditu.
- **KAFKA** — erdiko mezularitza-hub (topic-etan biltzen)
- **SINK CONNECTOR** — datuak Kafka-tik ateratzen ditu
- **SINKS:** sink-systems (Elasticsearch, HDFS, S3, MongoDB, BigQuery...)

Kafka Connect-eko osagaiak

- **Worker** → Kafka Connect exekutatzen duen prozesua
- **Connector** → zer sistema konektatu eta nola definitzen du
- **Task** → konektoreak egiten duen lana exekutatzen du
- **Plugin** → konektore mota eskaintzen duen softwarea

Worker + Connector konfigurazioa + Plugin-a = integrazioa

Worker konfigurazioa: connect-standalone.properties

```
connect-standalone.properties properties

1 # Kafka-broker-en bootstrap (Connect-ek mezuak hor bidaltzen)
2 bootstrap.servers=kafka1:19092
3
4 # Key eta Value converter-ak (JSON estandarra)
5 key.converter=org.apache.kafka.connect.json.JsonConverter
6 value.converter=org.apache.kafka.connect.json.JsonConverter
7
8 # Eskema-informazioa mezuetan barneratu
9 key.converter.schemas.enable=true
10 value.converter.schemas.enable=true
11
12 # Offset-jarraipena fitxategi-batean (standalone mode)
13 offset.storage.file.filename=/tmp/connect.offsets
14
15 # JAR-konektoreen kokapena (deskargatu eta hor ipini)
16 plugin.path=/opt/kafka/plugins
```

- **bootstrap.servers** → Kafka broker-a
- **converter** → mezuen formatua
- **offset.storage...** → non geratu den gogoratzeko
- **plugin.path** → konektoreen plugin-ak non dauden

— MYSQL SOURCE CONNECTOR

Source connector: mysql-source.properties (MySQL → Kafka)

mysql-source.properties properties

```
1 # Konektore-izena (kluster-mailan UNIKOA)
2 name=mysql-source
3
4 # Konektore-klasea - Confluent JDBC source
5 connector.class=io.confluent.connect.jdbc.JdbcSourceConnector
6
7 # MySQL-konexioa (retail_db datu-basea)
8 connection.url=jdbc:mysql://mysql:3306/retail_db
9 connection.user=iabd
10 connection.password=iabd
11
12 # Zein taula irakurri (whitelist edo blacklist)
13 table.whitelist=categories
14
15 # Detekzio-modua: incrementing (auto-increment-id-arabera)
16 mode=incrementing
17 incrementing.column.name=category_id
18
19 # Kafka-topic-aurrizkia (taula-bakoitzeko: aurrizki+taula-izena)
20 topic.prefix=iabd-retail_db-categories
```

categories taula irakurri



category_id berriak detektatu



Kafka topic-era bidali:

iabd-retail_db-categories

Sink Connector: mongodb-sink.properties (Kafka → MongoDB)

●●● mongodb_sink.properties properties

```
1 # Konektorearen izena
2 name=mongodb-sink
3
4 # Erabiliko den konektore-klasea
5 connector.class=com.mongodb.kafka.connect.MongoSinkConnector
6
7 # MongoDB-ra konektatzeko helbidea
8 connection.uri=mongodb://mongodb:27017
9
10 # Datuak zein datu-basetan eta zein bildumatan gordeko diren
11 database=iabd
12 collection=categories
13
14 # Sink konektoreak zein Kafka topic irakurriko duen
15 topics=iabd-retail_db-categories
```

iabd-retail_db-categories



MongoDB Sink



iabd.categories

— BI KONEKTOREAK BATERA: SOURCE + SINK

MySQL → Kafka → MongoDB bide osoa

MySQL → mysql-source → iabd-retail_db-categories → mongodb-sink → MongoDB

```
connect-both.sh

1 # Source + Sink konektoreak ALDI BEREAN abiarazi
2 # Standalone-mode-an, argumentu-anitz worker-bakar-batean
3
4 connect-standalone.sh \
5   config/connect-standalone.properties \
6   config/mysql-source-connector.properties \
7   config/mongodb-sink-connector.properties
8
9 # EMAITZA:
10 # 1. mysql-source -> MySQL-categories taula-tik mezuak
11 #    'iabd-retail_db-categories' topic-era
12 # 2. mongodb-sink -> 'iabd-retail_db-categories' topic-etik
13 #    mezuak MongoDB-bilduma-batean idatzi
14 #
```

Kafka Connect REST APIa — 8083 port-ean

■ Konektoreak:

```
curl http://localhost:8083/connectors
```

■ Status-a:

```
curl http://localhost:8083/connectors/mysql-source/status
```

```
curl http://localhost:8083/connectors/mongodb-sink/status
```

■ Espero dena:

RUNNING

Zer gertatzen da insert batekin?

- **MySQL-en:**

```
INSERT INTO categories  
(category_department_id, category_name)  
VALUES (2, 'Kirol berriak');
```



- **Source Connector-ek detektatu**



- **Kafka topic: iabd-retail_db-categories**



- **Sink Connector-ek kontsumitu**



- **MongoDB:**

```
{ "category_id": ...,  
  "category_department_id": 2,  
  "category_name": "Kirol berriak" }
```

MYSQL → KAFKA → MONGODB

HELBURUA: Kafka Connect erabiliz pipeline hau eraikitzea:

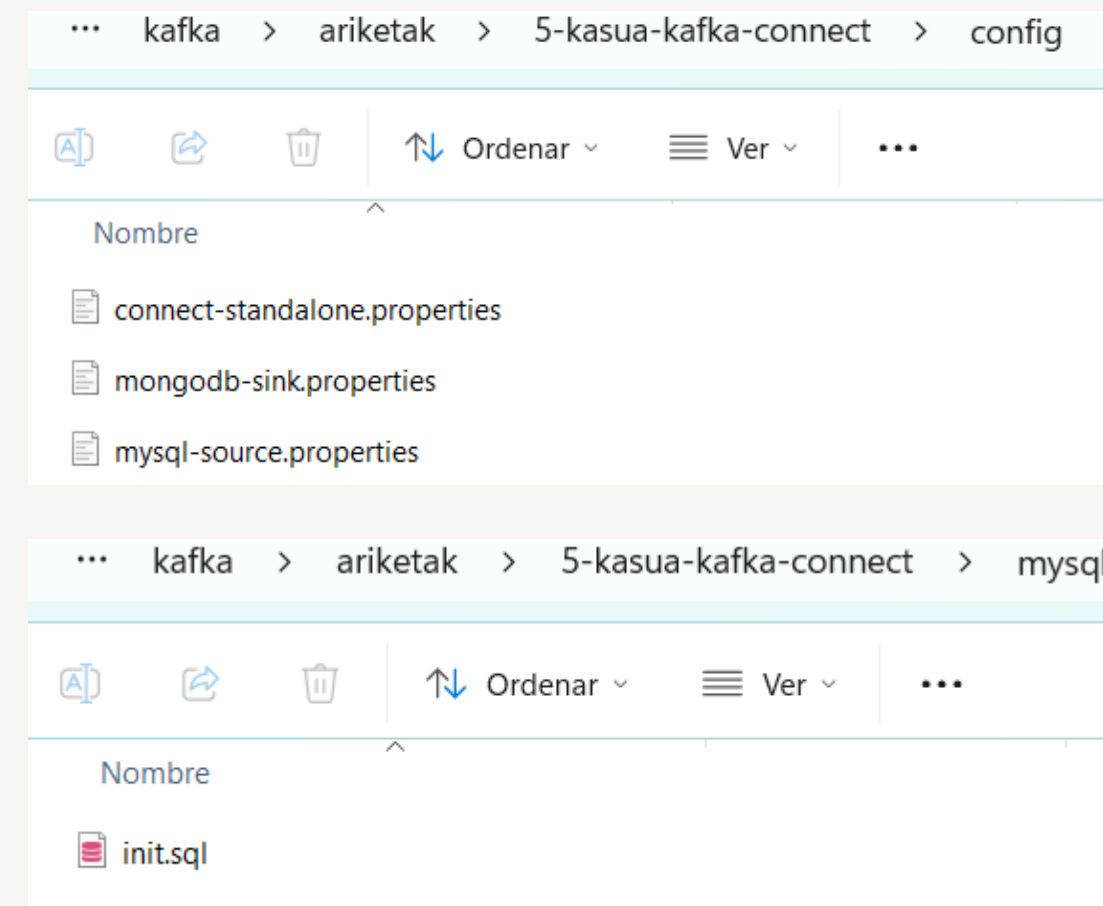
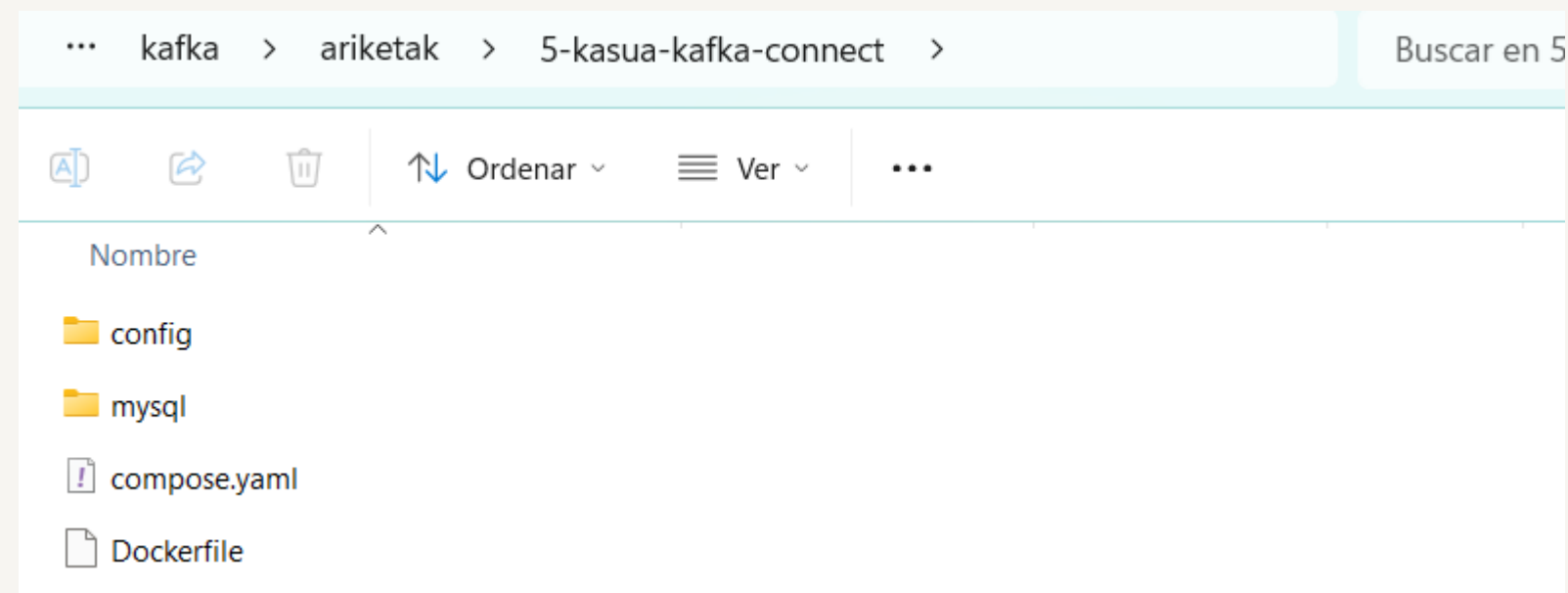


MySQL-eko **retail_db.categories** taulan sortutako erregistro berriak Kafka topic batera bidali eta automatikoki MongoDB-ko **iabd.categories** bilduman gordeko dira.

MYSQL → KAFKA → MONGODB

1. Sortu ingurunea:

Sortu ariketaren karpeta:



5. KASU PRAKTIKOA: MYSQL → KAFKA → MONGODB

MYSQL → KAFKA → MONGODB

Sortu **compose.yaml** fitxategia:

```
1 services:
2
3   # =====
4   # KAFKA
5   # =====
6   > Run Service
7   kafka:
8     image: confluentinc/cp-kafka:7.7.1
9     container_name: iabd-connect-kafka
10    hostname: kafka
11
12    ports:
13      - "9092:9092"
14
15    environment:
16      KAFKA_NODE_ID: 1
17
18      KAFKA_PROCESS_ROLES: broker,controller
19
20      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:29093
21
22      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
23
24      KAFKA_LISTENERS: INTERNAL://0.0.0.0:29092,EXTERNAL://0.0.0.0:9092,CONTROLLER://0.0.0.0:29093
25
26      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:29092,EXTERNAL://localhost:9092
27
28      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
29      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
30
31      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
32
33      CLUSTER_ID: MkU30EVBNTcwNTJENDM2Qk
34
35   # =====
36   # MYSQL
37   # =====
38   > Run Service
39   mysql:
40     image: mysql:8.4
41     container_name: iabd-connect-mysql
42
43     environment:
44       MYSQL_ROOT_PASSWORD: root
45       MYSQL_DATABASE: retail_db
46       MYSQL_USER: iabd
47       MYSQL_PASSWORD: iabd
48
49     ports:
50       - "3306:3306"
51
52     volumes:
53       - ./mysql/init.sql:/docker-entrypoint-initdb.d/init.sql
54
55   # =====
56   # MONGODB
57   # =====
58   > Run Service
59   mongodb:
60     image: mongo:8
61     container_name: iabd-connect-mongodb
62
63     ports:
64       - "27017:27017"
65
66   # =====
67   # KAFKA CONNECT
68   # =====
69   > Run Service
70   connect:
71     build: .
72     container_name: iabd-kafka-connect
73     hostname: connect
74
75     depends_on:
76       - kafka
77       - mysql
78       - mongodb
79
80     ports:
81       - "8083:8083"
82
83     volumes:
84       - ./config:/opt/kafka-connect/config
85
86     entrypoint:
87       - /bin/bash
88       - -c
89       - sleep infinity
```

Sleep infinity: container-a martxan mantentzen du, baina Kafka Connect worker-a ez du automatikoki abiarazten. Worker-a gero **connect-standalone** komandoarekin abiaraziko dugu.

MYSQL → KAFKA → MONGODB

compose.yaml fitxategia dagoen karpeta berean sortu:

Dockerfile (CMD: *New-Item Dockerfile -ItemType File*)

Eta jarri:

```
Dockerfile > ...
1 FROM confluentinc/cp-kafka-connect:7.7.1
2
3 USER root
4
5 # JDBC Source Connector
6 RUN confluent-hub install --no-prompt confluentinc/kafka-connect-jdbc:latest
7
8 # MongoDB Connector
9 RUN confluent-hub install --no-prompt mongodb/kafka-connect-mongodb:latest
10
11 # MySQL JDBC driver-a
12 RUN curl -L \
13     -o /usr/share/confluent-hub-components/confluentinc-kafka-connect-jdbc/mysql-connector-j.jar \
14     https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.4.0/mysql-connector-j-8.4.0.jar
15
16 USER appuser
```

Honek Kafka Connect irudiari **behar ditugun** bi **konektoreak** gehitzen dizkio:

- JDBC Connector (MySQL → Kafka)
- MongoDB Connector (Kafka → MongoDB)

MYSQL → KAFKA → MONGODB

Sortu: **mysql\init.sql**

Eduki honekin:

```
mysql > init.sql
1  USE retail_db;
2
3  CREATE TABLE categories (
4      category_id INT AUTO_INCREMENT PRIMARY KEY,
5      category_department_id INT NOT NULL,
6      category_name VARCHAR(45) NOT NULL
7  );
8
9  INSERT INTO categories
10 (category_department_id, category_name)
11 VALUES
12 (1, 'Football'),
13 (2, 'Basketball'),
14 (3, 'Running');
```

Horrela, ingurunea lehen aldiz abiaraztean MySQL-k automatikoki sortuko du: **retail_db.categories**

MYSQL → KAFKA → MONGODB

Ingurunea eraiki eta abiarazi:

- Lehen aldian, Kafka Connect-en plugin-ak instalatu behar direnez: **docker compose up -d --build**
- Egiaztatu: **docker ps**

Martxan egon behar dira: **Kafka + MySQL + MongoDB + Kafka Connect**

- iabd-connect-kafka
 - iabd-connect-mysql
 - iabd-connect-mongodb
 - iabd-kafka-connect
- Egiaztatu Kafka Connect prest dagoen: **curl http://localhost:8083**

MYSQL → KAFKA → MONGODB

2. Egiaztatu MySQL datuak

MySQL-en: `docker exec -it iabd-connect-mysql -uiabd -piabd retail_db`

```
USE retail_db;  
SELECT * FROM categories;
```

Identifikatu:

- `category_id`
- `category_department_id`
- `category_name`

MYSQL → KAFKA → MONGODB

3. Worker konfigurazioa:

Sortu fitxategia **config** karpetan:

Connect-standalone.properties

Osatu worker-aren konfigurazioa:

bootstrap.servers=...

key.converter=org.apache.kafka.connect.json.JsonConverter
value.converter=org.apache.kafka.connect.json.JsonConverter

key.converter.schemas.enable=true
value.converter.schemas.enable=true

offset.storage.file.filename=...

plugin.path=...

PISTAK:

- Kafka broker-a Docker sare barruan **kafka** izenez dago.
- Kafka-ren internal listener-a **29092** portuan dago.
- Standalone moduan offset-ak fitxategi batean gordetzen dira.
- Plugin-ak bi kokapen hauetan daude:
 - **/usr/share/java**
 - **/usr/share/confluent-hub-components**

MYSQL → KAFKA → MONGODB

4. Konfiguratu MySQL Source Connector

Sortu fitxategia:

mysql-source.properties

Konfigurazioak gutxienez izan behar ditu:

name=...
connector.class=...

connection.url=...
connection.user=...
connection.password=...

query=...

mode=incrementing
incrementing.column.name=...

topic.prefix=...

Horrela sortuko den topic-a:

iabd-retail_db-categories

MYSQL → KAFKA → MONGODB

5. Konfiguratu MongoDB Sink Connector

Sortu:

mongodb-sink.properties

Osatu:

name=...
connector.class=...

connection.uri=...

database=...
collection=...

topics=...

Helburua:

Kafka topic: **iabd-retail_db-categories**



MongoDB: **iabd.categories**

MYSQL → KAFKA → MONGODB

6. Abiarazi Kafka Connect (cd /opt/kafka-connect/config):

`connect-standalone connect-standalone.properties mysql-source.properties mongodb-sink.properties`

7. Egiaztatu konektoreak:

REST API (beste terminal batean):

`curl http://localhost:8083/connectors` (bi konektoreak agertu behar dira)

`curl http://localhost:8083/connectors/mysql-source/status`

`curl http://localhost:8083/connectors/mongodb-sink/status`

Egoera: RUNNING

MYSQL → KAFKA → MONGODB

8. Egiaztatu Kafka topic-a:

Consumer bat abiarazi:

```
kafka-console-consumer --topic iabd-retail_db-categories --from-beginning --bootstrap-server...
```

categories taulako datuak agertu behar dira.

9. Egiaztatu MongoDB:

MongoDB-n:

```
use iabd
```

```
db.categories.find()
```

MySQL-tik etorritako dokumentuak agertu behar dira.

MYSQL → KAFKA → MONGODB

10. Proba dinamikoa:

MySQL-en gehitu erregistro berri bat:

```
INSERT INTO categories (category_department_id, category_name)
VALUES (2, 'Streaming');
```

Itxaron segundo batzuk eta egiaztatu:

- Kafka topic-ean → erregistro berria agertu dela
- MongoDB-n ere bai:
 - `use iabd`
 - `db.categories.find({category_name: "Streaming"})`

MYSQL → KAFKA → MONGODB

10. Galderak:

1. Zein da **connect-standalone.properties** fitxategiaren funtzioa?
2. Zein da **mysql-source.properties** fitxategiaren funtzioa?
3. Zein da **mongodb-sink.properties** fitxategiaren funtzioa?
4. Zer esan nahi du **mode=incrementing** konfigurazioak?
5. Zein topic sortu da eta zer datu ditu?
6. Zer adierazten du MongoDB Sink Connector-eko topics parametroak?
7. Zein da Kafka-ren rola arkitektura honetan?
8. Zergatik ez dugu Python Producer eta Consumer programarik idatzi?
9. Idatzi datuek egiten duten ibilbide osoa.

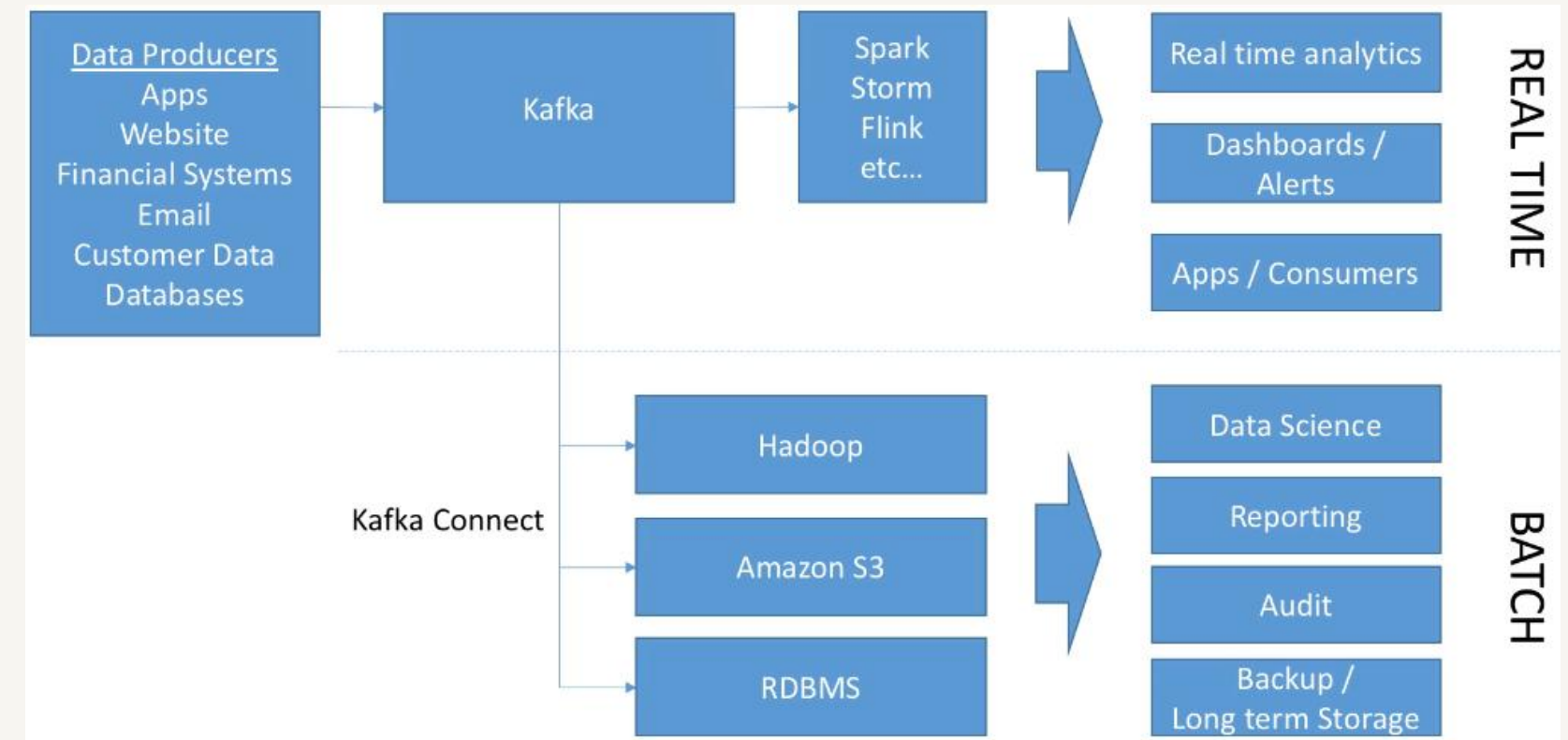
05

Kafka eta Big Data

— KAPPA ARKITEKTURA

Kafka Big Data arkitekturan — batch + streaming

- **KAFKA = MEZULARITZA-HUB ZENTRALA** — datu-iturri-anitzentzat
- **BATCH-INGEST eta STREAMING-INGEST BATERATUTA** — Kafka-bidez bietarako bide-bera
- **KAPPA ARKITEKTURA** — Lambda-aren-alternativa: STREAMING-FIRST, batch-bidearen-beharrik-gabe
- **BIG-DATA-EKOSISTEMA:** Kafka + Spark/Flink + data-lake (S3/HDFS) + DW (Snowflake/BigQuery)
- **Kafka Connect = inbiltze-PUNTU NAGUSIA** datu-iturri-anitzentzat



Gogoan hartu beharrekoak

-
- i* **Kafka klusterra** — 3 broker minimum produkzioan, broker-ID + port + log-dirs UNIKO-anitz konfigurazio-fitxategi-bana.

 - ii* **Partition + RF** — KLUSTERRA SORTU AURRETIK pentsatu (aldaketak ordena/errendimendua kaltetzen).

 - iii* **Medallion Kafka-en** — Bronze/Silver/Gold TOPIC-INDEPENDENTEAK; pipeline-modularizazioa + consumer-anitzen-aukera.

 - iv* **NiFi + Kafka** — PublishKafka + ConsumeKafka prozesadoreak; record-aldaerak schema-aware produkzio-best-practice.

 - v* **Kafka Connect** — kode-idatzi-gabe datu-iturri-anitz integrazioa; produkzio-ohikoa CDC-rako Debezium-ekin.

 - vi* **Kappa arkitektura** — STREAMING-FIRST eredua; Kafka + Spark/Flink + data-lake produkzio-estandar modernoa.

Galderak, zalantzak, iruzkinak.

Hurrengo saioa: DataFlow 5 — Spark Streaming.

APACHE KAFKA	kafka.apache.org/documentation/
CONNECT	#connect
DOKUMENTAZIOA	
CONFLUENT HUB —	hub.confluent.io
KONEKTORE- KATALOGO	
DEBEZIOUM CDC PLATFORM	debezium.io
KAPPA ARCHITECTURE (JAY KREPS)	kappa-architecture.com
JATORRIZKO MATERIALA	Aitor Medrano · CC BY-NC-SA